

Department: Computer Engineering & Applications

Programme: B.Tech

Branch: CSE

Subject Name: Application Build & Release

Automation Lab

Subject Code: BCSE0821

Assignment Instructions

- Submit all answers in 3 A4-size pages (handwritten) and attach 2 pages of screenshots.
- Write clearly and organize your answers properly, using headings where necessary.
- Ensure that all questions are attempted.
- Upload the assignment on or before the given deadline.

Assignment Question: A software company is developing a web application using a team of developers. The company wants to use Git to manage source code, track changes, enable parallel development, and support collaboration among team members. Three developers have been assigned different responsibilities: a Project Lead, a Feature Developer, and a Tester/Reviewer.

Based on this scenario, write a step-by-step procedure for designing and implementing a Git-based version control and collaborative development workflow. Explain how you would create and organize the Git repository, establish branches for different purposes, assign appropriate responsibilities to team members, use commits and pull requests, and manage version history.

Tasks

1. Repository Setup

- Create a Git repository for the project.
- Configure the repository with appropriate project files and a .gitignore.
- Configure Git username and email for each developer.

2. Branching Strategy

- Create and explain the purpose of branches such as:
 - main
 - develop
 - feature/*
 - bugfix/*
- Explain why developers should avoid directly modifying the main branch.

3. Role-Based Collaboration

- Define suitable responsibilities for the Project Lead, Feature Developer, and Tester/Reviewer.
- Explain how repository permissions can be configured so that only authorized users can perform operations such as merging or deleting protected branches.

4. Git Operations

- Demonstrate the procedure for:
 - Cloning a repository
 - Creating a branch
 - Making changes
 - Staging files
 - Creating commits
 - Pushing changes to the remote repository
 - Pulling the latest changes

5. Collaborative Development

- Explain how developers can work simultaneously on different features using separate branches.
- Explain how Pull Requests/Merge Requests can be used to integrate changes into the main development branch.

6. Version History and Rollback

- Explain how Git maintains a history of project changes.
- Demonstrate commands such as git log, git diff, and git revert.

- Explain how previous versions can be identified and restored safely.